L19 — Header Linked List and Doubly Linked List

Unit: 2 | **Chapter:** Linked Lists | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Describe header linked lists and their variants (grounded, circular)
- Implement a doubly linked list with insertion and deletion
- Explain why doubly linked lists are useful for bidirectional traversal
- Compare singly vs doubly linked list trade-offs

---

1. Header Linked List

A **header linked list** has a special **header node** (also called a sentinel or dummy node) at the beginning that does **not store data**. It simply marks the start of the list.

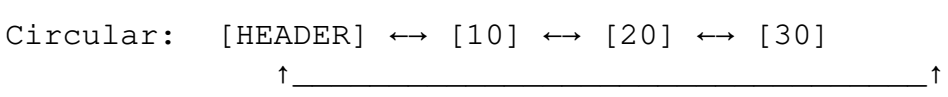
```
Without header:  HEAD → [10] → [20] → [30] → NULL

With header:     HEAD → [HEADER|] → [10] → [20] → [30] → NULL
                          ↑
                    Dummy node (data field unused)
```

1.1 Benefits of the Header Node

- Eliminates special cases for insertion/deletion at the beginning
- Operations are uniform — always traverse starting from `header.next`
- Makes algorithms cleaner, especially for deletion

1.2 Types of Header Lists

Type	Description
Grounded header list	Last node's `next` = NULL
Circular header list	Last node's `next` = header node



---

2. Header List Implementation

```
class Node:
    def __init__(self, data=None):
        self.data = data
```

```
        self.next = None

class HeaderLinkedList:
    def __init__(self):
        self.header = Node()   # Dummy header node
        self.header.next = None

    def insert_front(self, data):
        """Insert after header — always O(1), no special cases"""
        new_node = Node(data)
        new_node.next = self.header.next
        self.header.next = new_node

    def delete(self, key):
        """Delete node with key — uniform traversal from header"""
        prev = self.header     # Start from header (never None)
        curr = self.header.next
        while curr:
            if curr.data == key:
                prev.next = curr.next
                return True
            prev = curr
            curr = curr.next
        return False

    def display(self):
        curr = self.header.next
        while curr:
            print(curr.data, end=" → ")
            curr = curr.next
        print("NULL")
```

With the header node, we never need to check `if prev is None` — `prev` is always at least the header.

---

3. Doubly Linked List

A **doubly linked list (DLL)** has nodes with two pointers:

- `prev` → points to the previous node
- `next` → points to the next node

```
NULL ← [PREV|10|NEXT] ↔ [PREV|20|NEXT] ↔ [PREV|30|NEXT] → NULL
         ↑
        HEAD

class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

4. Doubly Linked List Operations

4.1 Insert at Beginning — O(1)

```
def insert_beginning(self, data):
    new_node = DNode(data)
    new_node.next = self.head
    if self.head:
        self.head.prev = new_node
    self.head = new_node
```

4.2 Insert at End — O(n) or O(1) with tail pointer

```
def insert_end(self, data):
    new_node = DNode(data)
    if self.head is None:
        self.head = new_node
        return
    curr = self.head
    while curr.next:
        curr = curr.next
    curr.next = new_node
    new_node.prev = curr
```

4.3 Insert After a Specific Node — O(1)

```
def insert_after(self, node, data):
    if node is None:
        return
    new_node = DNode(data)
    new_node.next = node.next
    new_node.prev = node
    if node.next:
        node.next.prev = new_node
    node.next = new_node
```

4.4 Delete a Node — O(1) given node reference

This is the major advantage of DLL over SLL: deletion without needing the previous node.

```
def delete_node(self, node):
    """Delete a specific node — O(1) since we have prev pointer."""
    if node is None:
        return

    if node.prev:
        node.prev.next = node.next   # Link prev to next
    else:
        self.head = node.next        # Deleting head

    if node.next:
```

```
        node.next.prev = node.prev   # Link next to prev
```

4.5 Forward and Backward Traversal

```
def traverse_forward(self):
    curr = self.head
    while curr:
        print(curr.data, end=" ⇒ ")
        curr = curr.next
    print("NULL")

def traverse_backward(self):
    """Find tail first, then traverse backward."""
    curr = self.head
    if curr is None:
        return
    while curr.next:
        curr = curr.next
    # curr is now at tail
    while curr:
        print(curr.data, end=" ⇒ ")
        curr = curr.prev
    print("NULL")
```

---

5. Complete DLL Demo

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new = DNode(data)
        if not self.head:
            self.head = new
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new
        new.prev = curr

    def delete_by_value(self, key):
        curr = self.head
        while curr:
            if curr.data == key:
                if curr.prev:
                    curr.prev.next = curr.next
                else:
                    self.head = curr.next
                if curr.next:
                    curr.next.prev = curr.prev
```

```
                return
            curr = curr.next


dll = DoublyLinkedList()
for v in [10, 20, 30, 40]:
    dll.append(v)

dll.traverse_forward()   # 10 ⇒ 20 ⇒ 30 ⇒ 40 ⇒ NULL
dll.delete_by_value(20)
dll.traverse_forward()   # 10 ⇒ 30 ⇒ 40 ⇒ NULL
dll.traverse_backward()  # 40 ⇒ 30 ⇒ 10 ⇒ NULL
```

---

6. Singly vs Doubly Linked List

Feature	Singly (SLL)	Doubly (DLL)
Memory per node	1 pointer	2 pointers
Backward traversal	Not possible	O(n)
Delete given node ref	O(n) — need prev	**O(1)**
Reverse	O(n)	O(n) but easier
Implementation complexity	Simpler	More complex

Use DLL when: backward traversal needed, or O(1) deletion given a node reference (e.g., LRU cache, deque).

Summary

- **Header linked list:** dummy header node eliminates special cases; circular variant has last→header.
- **Doubly linked list:** each node has `prev` and `next` pointers.
- DLL enables **bidirectional traversal** and **O(1) deletion** given a node reference.
- Extra memory cost: one additional pointer per node (8 bytes on 64-bit systems).

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)